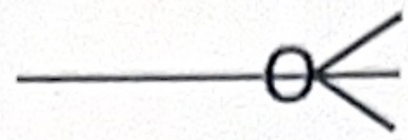
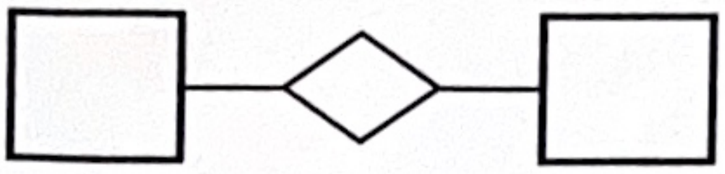
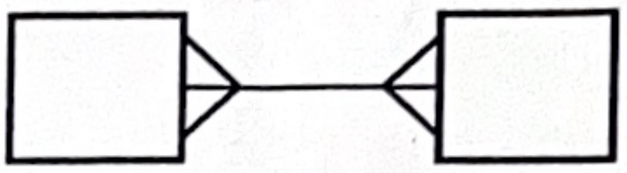
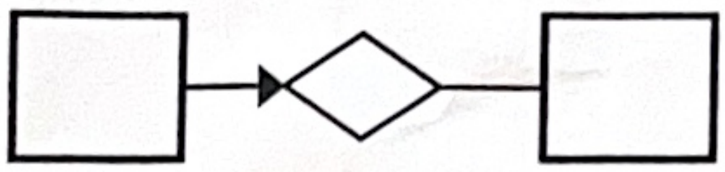
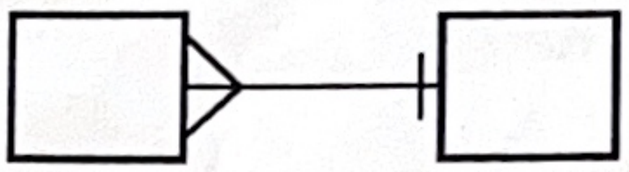
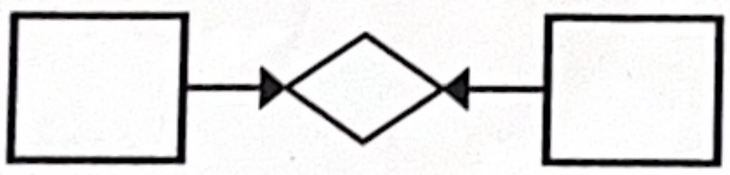
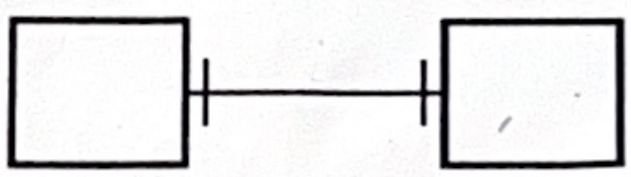
Relationship Cardinality/Constraints

	Chen's notation	Crow's foot notation
Optional Many 0..m		
Mandatory Many 1..m		
Optional One 0..1		
Mandatory One 1..1		

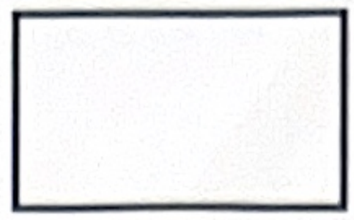
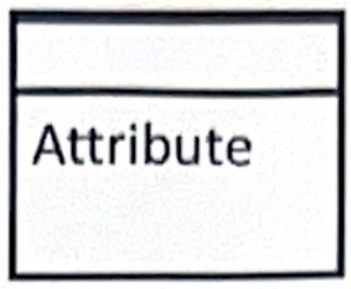
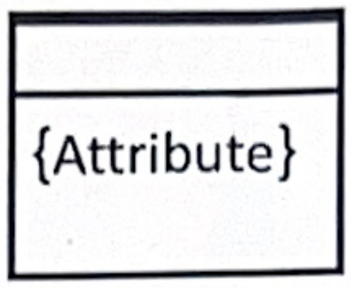
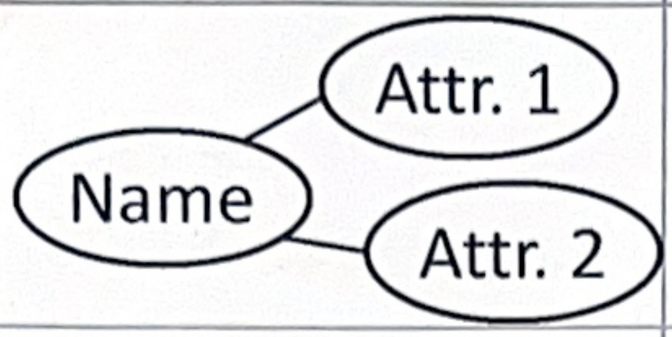
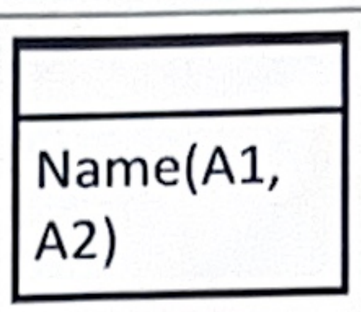
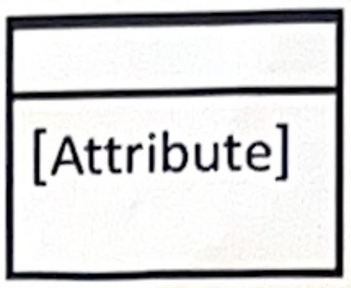
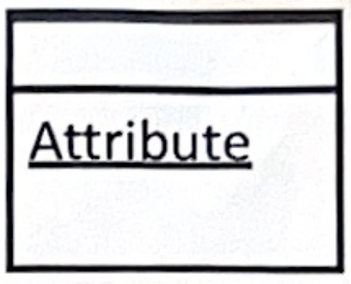
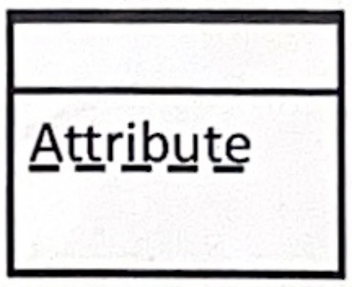
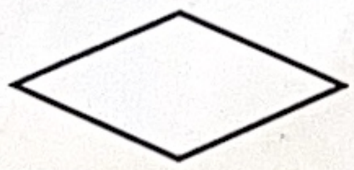
BINARY Relationship Cardinalities

Here we have just looked at cardinalities and omitted participation constraints (optional/mandatory) for clarity

Many to Many		
One to Many *example:*		
One to One		

Conceptual Model Mapping

Note: This table shows how to draw **conceptual** models. Do not use the notation marked with * in **physical** models.

Concept	Chen's notation	Crow's foot notation	
Entity			
Weak Entity			
Attribute			
Multivalued A.			*
Composite A.			*
Derived A.			*
Key A.			*
Weak (or Partial) Key A.			*
Relationship			
Weak (Identify-ing) Relationship			

Formula sheet

Cost of Joins

Equality join	Equality Join, One Column (R ⋈ S) • SELECT * FROM Reserves R1, Sailors S1 WHERE R1.sid=S1.sid • Cost metric : Number of pages; Number of I/O • Example: R has 6 rows, S has 6, total 6x6=36
Simple Nested Loop Join (SNLJ)	Cost (SNLJ) = NPages(Outer) + Ntuples (Outer) * NPages(Inner)
Page-Oriented Nested Loops Join (PNLJ)	Cost (PNLJ) = NPages(Outer) + NPages(Outer)*NPages(Inner) 和 SNLJ 不一样的是 Ntuple 变成了 Npages
Block Nested Loops Join (BNLJ)	Cost (BNLJ) = NPages(Outer) + NBlocks(Outer)*NPages(Inner) **NBlocks(Outer)** = $\left\lceil \frac{NPages(Outer)}{B-2} \right\rceil$ **B = # pages of space in memory, i.e., Blocks** B is number of buffer pages
Sort-Merge Join	Cost (SMJ) = sort(Outer) + sort(Inner) + Npages(Outer) + Npages(Inner) Sort(R) = External sort cost = 2 * NumPasses * Npages(R)
Hash Join	Cost(HJ) = 2 * Npages(Outer) + 2 * Npages(Inner) + Npages(Outer) + Npages(Inner)
Tuple-oriented Nested Loops Join cost	Tuple-oriented Nested Loops Join cost = Npages(R) + Ntuples (R) * Npages(S)

Result size

1. Single table selection:

   ResultSize = $NTuples(R) \prod_{i=1..n} RF_i$

2. Joins (over k tables):

   ResultSize = $\prod_{j=1..k} NTuples(R_j) \prod_{i=1..n} RF_i$

Reduction Factors

Col = value

RF = 1/NKeys(Col)

(2) Col > value

RF = (High(Col) – value) / (High(Col) – Low(Col))

(3) Col < value

RF = (val – Low(Col)) / (High(Col) – Low(Col))

(4) Col_A = Col_B (for joins)

RF = 1/ (Max (NKeys(Col_A), NKeys(Col_B)))

(5) if no information about Nkeys or interval, use a "magic number" 1/10

RF = 1/10

Cost of index selections

1. Sequential (heap) scan of data file:

Cost = Npages(R)

2. Index selection over a primary key (just a single tuple) 如: studentid = 1461821:

- Cost(B+Tree)=Height(Index)+1, Height is the index height

Cost(HashIndex)= ProbeCost(Index)+1, ProbeCost(I)~1.2

3. Clustered index matching one or more predicates 如: studentid > 1461821:

$$Cost(B+Tree)=(NPages(I) + NPages(R))*\prod_{i=1..n} RF_i$$

$$Cost(HashIndex)= NPages(R)*\prod_{i=1..n} RF_i * 2.2$$

4. Non-clustered index matching one or more predicates 如: studentid > 1461821 :

$$Cost(B+Tree)=(NPages(I) + NTuples(R))*\prod_{i=1..n} RF_i$$

$$Cost(HashIndex)= NTuples(R)*\prod_{i=1..n} RF_i * 2.2$$

Cost of Alternative plan

Let's say that Sailors(S) has 500 pages, 40000 tuples, NKeys(rating) = 10

```
SELECT S.sid FROM Sailors S WHERE S.rating=8
```

Result size = (1/NKeys(rating)) * NTuples(S) = (1/10)*40000 =4000 tuples

1. First plan : If we have I(rating), NPages(I) = 50

   (a) Clustered index:

   Cost = (1/NKeys(rating))*(NPages(I)+NPages(S))=(1/10)*(50+500) = 55 I/O (选这个！)

   (b) Unclustered index:

   Cost = (1/NKeys(rating))*(NPages(I)+NTuples(S))=(1/10)*(50+40000) = 4005 I/O

2. Second plan: If we have an Index(sid), NPages(Index)= 50:
   - Would have to retrieve all tuples/pages.

   (a) clustered index, the cost is 50+500

   (b) unclustered index, 50+40000

3. Third plan: Doing a file scan:

   - Cost = NPages(S) = 500

1 MB megabyte = 1000000 bytes

1 KB kilobyte ≈ 1000 bytes

1 GB gigabytes ≈ 1,000,000,000 bytes.

Week 5 lec 1

INSERT

→ Insert records from a table

```
INSERT INTO NewEmployee
    SELECT * FROM Employee;
```

→ Insert multiple records

All columns must be inserted

```
INSERT INTO Employee VALUES
    (DEFAULT, "A", "A's Addr", "2012-02-02", NULL, "S"),
    (DEFAULT, "B", "B's Addr", "2012-02-02", NULL, "S"),
    (DEFAULT, "C", "C's Addr", "2012-02-02", NULL, "S");
```

Specific columns will be inserted

```
INSERT INTO Employee
    (Name, Address, DateHired, EmployeeType)
    VALUES
        ("D", "D's Addr", "2012-02-02", "C"),
        ("E", "E's Addr", "2012-02-02", "C"),
        ("F", "F's Addr", "2012-02-02", "C");
```

hint: if MySQL complains, use 'single quotes'

UPDATE

→ Change existing data in tables
→ Orders of statements is important
→ Specify a WHERE clause is important, by default, it will operate on the whole table/column
❖ For example:

// increase all salaries by 10%

```
UPDATE Hourly
    SET HourlyRate = HourlyRate * 1.10;
```

// increase all salaries greater than $100000 by 10% and all other salaries by 5%

```
UPDATE Salaried
    SET AnnualSalary =
        CASE
            WHEN AnnualSalary <= 100000
            THEN AnnualSalary * 1.05
            ELSE AnnualSalary * 1.10
        END;
```

// salary is lower than 100000 increase it by 5%, otherwise increase it by 10%

REPLACE

→ REPLACE works identically as INSERT, except if an old row has a key value that is same as the new row, then it is overwritten.

❖ For example:

Assume we have:

	engid	name	address	DateHired	EmployeeType
	1	Bob	123 Fake St	2012-02-02	S
2	2	Joe	22 Wallaby Way	2012-02-03	C
	3	Sally	4 Flinders St	2012-02-04	S
	4	Kayla	2 Synergy St	2012-02-05	S
	5	Erina	88 Innovation Dr	2012-02-06	C

```
REPLACE INTO Employee VALUES
    (DEFAULT, 'Mei', '3 Christmas Road', '2012-12-25', 'S');
```

6	Mei	3 Christmas Road	2012-12-25	S

```
REPLACE INTO Employee VALUES
    (2, 'Joe 2.0', '99 Wallaby Way', '2013-1-1', 'S');
```

2	Joe 2.0	99 Wallaby Way	2013-01-01	S

DELETE

→ Delete all records unless we use WHERE
→ Be aware of the foreign key constraints (ON DELETE CASCADE; ON DELETE RESTRICT)
❖ For example:

```
DELETE FROM Employee
    WHERE Name = "Grace";
```

Week 5 lec 2

Heap files VS Sorted files VS Index files

Heap files: no order among records
- Suitable when typical access is a file scan retrieving all records

Sorted Files: pages and records within pages are ordered by some condition
- The sorted file records are placed in some sequential order based on search key.
- Best for retrieval (of a range of records) in some order

Indexes
- Definition: An index is a data structure built on top of data pages used for efficient search.
- Any subset of the fields of a table is indexed based on **queries that are frequently run** against the database.
- The index is built over specific fields called search key fields. The index speeds up selections on the search key fields
- Note: Search key is not the same as key (e.g. doesn't have to be unique). (e.g., GPA, which is "not" a PK!)
- Indexes are stored in an index file
- Index can be of different types: clustered, unclustered, primary, secondary
  - Clustered index: data records in the data file have the same order as data entries of the index
  - Unclustered: data records in the data file are not sorted by search key/data entries
  - Primary: data is stored based on the primary key of the relation
  - Secondary: data is stored based on multiple attributes as indexes to improve the search performance

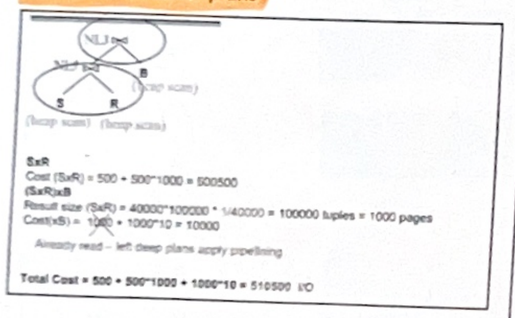
Primary vs Secondary index?
Consider
- Primary key can be used in the cases where the records are retrieved based on the value of primary key, otherwise, the secondary indexes should be preferred using fields that are frequently used in the queries

Cluster vs Unclustered index
- A query consists of a condition to check for a range: a clustered index is preferred as compared to unclustered. However, for equality conditions it does not have any advantage over the other if search key does not have duplicate values.
- Clustered index are expensive as the update/delete/insert might lead to re-odering of the records. Hence, index should only be clustered if there is a frequently-executed query

Hash vs Tree indexes
- Many range queries: B-tree index
- Equality queries: hash indexes (B-tree will increase the speed of most equality queries, but hash indexes are even faster)

Week7 lec 2

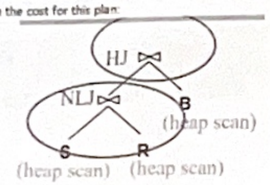
Estimate the cost of plans



SxR
Cost (SxR) = 500 + 500*1000 = 500500
(SxR)xB
Result size (SxR) = 40000*100000 * 1/40000 = 100000 tuples = 1000 pages
Cost(xB) = 1000 + 1000*10 = 10000
Already read – left deep plans apply pipelining

Total Cost = 500 + 500*1000 + 1000*10 = 510500 I/O

Estimate the cost for this plan:



SxR
Cost (SxR) = 500 + 500*1000 = 500500
(SxR)xB
Result size (SxR) = 100000*40000 *1/40000 = 100000 tuples = 1000 pages
Cost(xB) = 3*1000 + 3*10 = 2*1000 + 3*10 = 2030
2
Already read once – left deep plans apply pipelining

Total Cost = 500 + 500*1000 + 2*1000+ 3*10 = 502530 I/O

Week8 lec1 & 2 - Normalization

Problems with Denormalized Data

Student-ID	Course-ID	Fee
130	C200	75
200	C300	100
250	C200	75
425	C400	150
500	C300	100
575	C500	50
...	...	...

1. **Insertion Anomaly:** A new course cannot be added until at least one student has enrolled (which comes first student or course?)
2. **Deletion Anomaly:** If student 425 withdraws, we lose all record of course C400 and its fee!
3. **Update Anomaly:** If the fee for course C200 changes, we have to change it in multiple records (rows), else the data will be inconsistent.

Functional Dependency: Definitions
- Determinants (X,Y→ Z)
  – the attribute(s) that can determine the other attribute, which are the attributes on the left hand side of the arrow
- Key and Non-Key attributes
  – The key must be minimal, a key is a set of attributes for a relation R in form {X, Y ...}, such that the key determines the rest of the attributes of R. each attribute is either part of the primary key or it is not.
- Partial functional dependency (Y→ Z)
  – a partial functional arises when one or more non-key attributes are functionally determined by a subset of the primary key. A functional dependency of one or more non-key attributes upon part (but not all) of the primary key
- Transitive dependency (Z→ D)
  – when a non-key attribute is determined by another non-key attribute, such a dependency is called transitive dependency. A functional dependency between 2 (or more) non-key attributes

Armstrong's Axioms

1. Reflexivity:

$$B \subseteq A \Longrightarrow A \rightarrow B$$

Example: Student_ID, name -> name
If A and B are both the subset of attributes of R, and B is a subset of A, then with A, we can determine B

2. Augmentation:

$$A \rightarrow B \Longrightarrow \mathrm{AC} \rightarrow BC$$

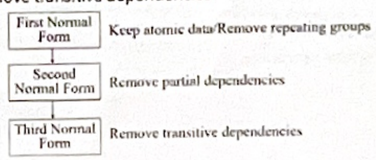
Example: Student_ID -> name => Student_ID, surname ->name, surname

3. Transitivity:

$$A \rightarrow B \text{ and } \mathrm{B} \rightarrow C \Longrightarrow A \rightarrow C$$

Example: ID -> birthdate and birthdate -> age then ID ->age

Steps in Normalisation
1. Keep atomic data/Remove repeating groups
2. Remove partial dependencies
3. Remove transitive dependencies



1. First Step: First Normal Form, remove repeating Groups

- repeating groups of attributes cannot be represented in a flat, two dimensional table
- removing attributes with multiple values (keep atomic data保留原子数据)
- Example:

不是拆掉特别多行 e.g LoanStatus (LoanNum(FK), DuePaid(PK), Status)

Set of values

Example Order-Item (Order#, Customer#, (Item#, Desc, Qty))

- Order-Item (Order#, Customer#, (Item#, Desc, Qty))
- Order-Item (Order#, Item#, Desc, Qty)
- Order (Order#, Customer#)

Break them into two
Use PK/FK to connect

2. Second step: Second Normal Form, Remove Partial Dependencies

- a non-key attribute cannot be identified by part of a composite key
- Example

should not

Example Order-Item (Order#, Item#, Desc, Qty)

- Order-Item (Order#, Item#, Desc, Qty)

Partial dependency

- Item (Item#, Desc)
- Order-Item (Order#, Item#, Qty)

改完partial dependency之后

- Partial Dependency Anomalies/Issues

Order-Item (Order#, Item#, Desc, Qty)

Order#	Item#	Desc	Qty
27	671	nail	2
28	402	bolt	1
28	671	nail	10
30	495	washer	50

- *UPDATE* change item desc in many places
- *DELETE* data for last item lost when last order for that item is deleted
- *INSERT* cannot add new item until it is ordered

Order-Item

Order#	Item#	Qty
27	671	2
28	402	1
28	671	10
30	495	50

To delete last order for item

Item

Item#	Desc
671	nail
402	bolt
495	washer

Add new item at any time

change item description in one place

3. Third Step: Third Normal Form, Remove Transitive Dependencies

- a non-key attribute cannot be identified by another non-key attribute
- Example

should not

Example Employee (Empl#, Ename, Dept#, Dname)

- Employee (Empl#, Ename, (Dept#, Dname))

Transitive dependency

- Employee (Empl#, Ename, Dept#)
- Department(Dept#, Dname)
- 为什么employee id也不是department的key，所以department 是 transaitive dependency

解决/删除 Transitive Dependency后

Employee

Empl#	Ename	Dept#
10	Smith	D1
20	Jones	D7
25	Smith	D7
30	Black	D8

To delete last empl in dept, but dept remains

Dept#	Dname
D1	MIS
D7	Finance
D8	Sales

To add new dept at any time

To change dept name in one place

Normalisation vs Denormalization

Normalisation:

– Normalised relations contains a minimum amount of redundancy and allow users to insert, modify, and delete rows in tables without errors or inconsistencies (anomalies)

Denormalization:

– The pay-off: query speed.
– The price: extra work on updates to keep redundant data consistent.
• Denormalization may be used to improve performance of time-critical operations.

Week9 lec1 Transactions

Transaction Properties ACID

1. Atomicity
   - A transaction is treated as a single, indivisible, logical unit of work. All operations in a transaction must be completed, if not, then the transaction is aborted, and everything is then undone
2. Consistency
   - Constraints that hold before a transaction must also hold after the transaction
   - multiple users accessing the same data see the same value
3. Isolation
   - Changes made during execution of a transaction cannot be seen by other transactions until this one is completed
4. Durability
   - When a transaction is complete, the changes made to the database are permanent, even if the system fails

Concurrent access

1. The Lost Update problem

   第一个用户打开了数据，然后更改数据，第二个用户在这个打开和更改间隙打开了数据库，因此看到的和即修改的数据都是未经更改的。

2. The Uncommitted Data problem

   未上传数据

   - Uncommitted data occurs when two transactions execute concurrently and the first is rolled back after the second has already accessed the uncommitted data.
   - 第一个用户打开了数据，然后更改数据，第二个用户在这个打开和更改间隙打开了数据库，并用第一个用户更改后的数据进行操作，但此时第一个用户未能复原数据，这时，第二个用户其实应用错数据。

3. The Inconsistent Retrieval problem

   不一致检索

   - Occurs when one transaction calculates some aggregate functions over a set of data, while other transactions are updating the data. Some data may be read after they are changed and some before they are changed, causing inconsistent results.
   - 第一个用户正在查看table的所有数据，而第二个用户刚好在更改table里的数据，导致table前半部分的数据是原始数据，后半部分变成了更改后的数据，导致数据的不一致性。
   - Example:

Alice	Bob
SELECT SUM(Salary) FROM Employee;	UPDATE Employee SET Salary = Salary * 1.01 WHERE EmpID = 33;
	UPDATE Employee SET Salary = Salary * 1.01 WHERE EmpID = 44;
(finishes calculating sum)	COMMIT;

Concurrency control methods

Lock

1. Locking (the main one we use)
2. Time stamping
3. Optimistic Concurrency Control

a. Database-level lock
   - Entire database is locked
   - Good for batch processing but unsuitable for multi-user DBMSs
   - T1 and T2 cannot access the same database concurrently even if they use different tables

b. Table-level lock
   - Entire table is locked - as above but not quite as bad
   - T1 and T2 can access the same database concurrently as long as they use different tables
   - Even if transactions want to access different parts of the table and would not interfere with each other
   - Not suitable for highly multi-user DBMSs

c. Page-level lock
   - An entire disk page is locked
   - Not commonly used now

4. Row-level lock
   - Allows concurrent transactions to access different rows of the same table, even if the rows are located on the same page
   - Improves data availability but with high overhead (each row has a lock that must be read and written to)
   - Currently the most popular approach (MySQL, Oracle)

5. Field-level lock
   - Allows concurrent transactions to access the same row, as long as they access different attributes within that row
   - Most flexible lock but requires an extremely high level of overhead
   - Not commonly used

7. Binary Locks
   - Has only two states: locked (1) or unlocked (0)
   - Eliminates "Lost Update" problem
     - the lock is not released until the statement is completed
   - Maybe too restrictive to yield optimal concurrency, as it locks even for two READs (when no update is being done)

Alternative concurrency control methods

1. Timestamp
   - Assigns a global unique timestamp to each transaction
   - Each data item accessed by a transaction gets the timestamp
   - Thus for every data item, the DBMS knows which transaction performed the last read or write on it
   - When a transaction wants to read or write, the DBMS compares its timestamp with the timestamps already attached to the item and decides whether to allow access

2. Optimistic
   - Based on the assumption that the majority of database operations do not conflict
   - Transaction is executed without restrictions or checking
   - Then when it is ready to commit, the DBMS checks whether any of the data it read has been altered – if so, rollback

Week9 lec 2

Categories of Failures

1) Statement failure
   - Syntactically incorrect
2) User process failure
   - The process doing the work fails (error occurs, dies)
3) Network failure
   - Network failure between the user and the database
4) User error
   - User accidentally drops the rows, table, database
5) Memory failure
   - Memory fails, becomes corrupt
6) Media failure
   - Disk failure, corruption, deletion

Types of Backups

- Physical vs Logical
- Online vs Offline
- Full vs Incremental
- Onsite vs Offsite

Physical backup

- raw copies of files and directories
- suitable for large databases that need fast recovery
- database is preferably offline ("cold" backup) when backup occurs 至关重
  - MySQL Enterprise automatically handles file locking, so database is not wholly off line
- backup is the exact copies of the database directories and files
- backup should include logs
- backup is only portable to machines with a similar configuration With own database 的公司不适合
- to restore
  - shut down DBMS
  - copy backup over current structure on disk
  - restart DBMS

Logical backup

- backup completed through SQL queries
- slower than physical
  - SQL Selects rather than OS copy
- output is larger than physical, doesn't include log or config files
- machine independent
- server is available during the backup
- in MySQL can backup using
  - Mysqldump
  - SELECT ... INTO OUTFILE
- to restore 倒拿一点
  - Use mysqlimport, or LOAD DATA INFILE within the mysql client

Online (or HOT) backup

- backups occur when the database is "live" 面对客户的公司适合,
- clients don't realise a backup is in progress
- need to have appropriate locking to ensure integrity of data

Offline (or COLD) backup

- backups occur when the database is stopped
- to maximize availability to users, take backup from replication server not live server
- simpler to perform
- cold backup is preferable, but not available in all situations
  e.g. applications without downtime

Full

- a full backup is where the complete database is backed up
  - may be Physical or Logical, Online or Offline
- it includes everything you need to get the database functional in the event of a failure

Incremental

- only the changes since the last backup are backed up
- for most databases this means only backup log files
- to restore:
  - stop the database, copy backed up log files to disk
  - start the database and tell it to redo the log files

Offsite Backup

- Enables disaster recovery (because backup is not physically near the disaster site)
- Example solutions:
  - → backup tapes transported to underground vault
  - → remote mirror database maintained via replication
  - → backup to Cloud

Week 10 lec 1

Difference between Transactional & Informational Systems

Characteristic	Transactional	Informational
Primary Purpose	Run the day to day business	Support decision making
Type of Data	Current data – representing the state of the business	Historical data – snapshots and predictions
Primary Users	Customers, clerks and other employees	Managers, analysts
Scope of Usage	Narrow, planned, fixed interfaces	Broad, ad hoc, complex interfaces
Design Goal	Performance and availability	Flexible use and data accessibility
Volume	Many constant updates and queries on a few tables or rows	Periodic batch updates, complex querying on multiple or all rows.
Example of Queries	What is the membership number of a customer who has phone number of 5551212? Select membership_nbr from MEMBER_INDEX where phone_num = '555-1212'	Campaign Management: How many customers purchased more than $500 worth of alcohol in our Melbourne stores this year?

Characteristics of a Database warehouse

1. Subject oriented
   - Data warehouses are organised around particular subjects (sales, customers, products)
2. Validated, integrated data
   - Data from different systems converted to a common format allows comparison and consolidation of data from different sources
   - Data from various sources validated before storing it in a data warehouse
3. Time variant
   - Historical data
   - Trend analysis crucial for decision support: requires historical data
   - Data consists of a series of "snapshots" which are time stamped
4. Non volatile
   - Users have read access only – all updating done automatically by ETL process and periodically by a DBA

Dimensional Modelling

- Popularised by Ralph Kimball in the 1990s, also known as star schema design.
- A dimensional model consists of:
  - Fact table
  - Several dimensional tables
  - (Sometimes) hierarchies in the dimensions
- It is essentially a simple and restricted type of ER model.

Distributed Database

- a single logical database physically spread across multiple computers in multiple locations that are connected by a data communications link.

- Objectives / goals of distributed database:
  1- Location transparency, a user does not need to know where particular data are stored

  2- Local autonomy, a node can continue to function for local users if connectivity to the network is lost

- appears to users as though it is one database

Advantages of distributed DBMS

1. Good fit for geographically distributed organizations / users
2. Data located near site with greatest demand
   E.g. ESPN Weekend Sports Scores
   - AFL - Melbourne
   - EPL – London
   - NFL – New York
   - Hurling - Dublin
3. Faster data access (to local data)
4. Faster data processing
   - Workload split amongst physical servers
5. Allows modular growth, can add new servers as load increases (horizontal scalability)
6. Increased reliability and availability
   - less danger of a single-point of failure (SPOF), IF data is replicated

Disadvantages of distributed DBMS

Complexity of management and control
- Database or/and application must stitch together data across sites
- Who and where is the current version of the record (row & column)?
- Who is waiting to update that information and where are they?
- How does the logic display this to the web & application server?

Data integrity
- Additional exposure to improper updating
- If two users in two locations update the record at the exact same time, who decides which statement should "win"?
- Solution: Transaction Manager or Master-slave design

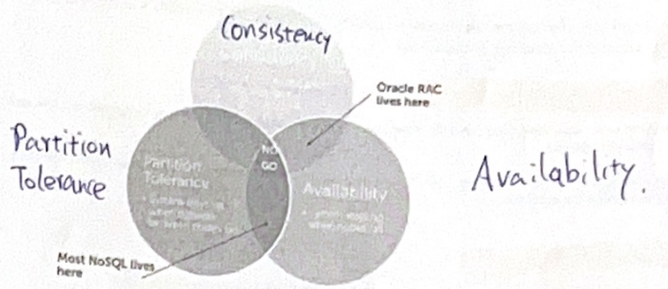
Security
- Many server sites -> higher chance of breach
- Multiple access sites require protection including network and storage infrastructure from both cyber & physical attacks

Trade-offs when dealing with DDBMS

- Availability vs Consistency
  - The CAP theorem says we need to decide whether to make data **always available** OR **always consistent**
- Synchronous vs Asynchronous updates
  - Are changes immediately visible everywhere (great BUT expensive) or later propagated (less expensive, but seeing stale data)?

The CAP theorem

- The CAP (Brewer's) Theorem states that you can only have two out of three of Consistency, Partition Tolerance, Availability



Synchronous updates

- Data is continuously kept up to date
  - users anywhere in the world can access data and get the same answer
- If any copy of a data item is updated anywhere on the network, the same update is immediately applied to all other copies or it is aborted.
- Ensures data integrity and minimizes the complexity of knowing where the most recent copy of data is located
- Can result in *slow response time* and *high network usage*
  - the DDBMS spends time checking that an update is accurately and completely propagated across the network.
  - The committed updated record must be identical in all servers

Asynchronous updates

- Some delay in propagating data updates to remote databases
  - some degree of at least temporary inconsistency is tolerated
  - may be ok if it is temporary and well managed
- Acceptable response time
  - updates happen locally and data replicas are synchronized in batches and predetermined intervals
- May be more complex to plan and design
  - need to ensure the right level of data integrity and consistency
- Suits some information systems more than others
  - compare commerce/finance systems with social media

NoSQL database properties

- Features
  - Doesn't use relational model or SQL language
  - Runs well on distributed servers
  - Most are open-source
  - Built for the modern web
  - Schema-less (though there may be an "implicit schema")
  - Supports schema on read
  - Not ACID compliant
  - 'Eventually consistent'

  High availability, scalability, performance
- Goals
  - to improve programmer productivity (OR mismatch)
  - to handle larger data volumes and throughput (big data)

Types of NoSQL databases

1. Document
2. Column - family
3. Graph
4. Key - value

Types of NoSQL: key-value stores

- Key = primary key
  Value = anything (number, array, image, JSON) –the application is in charge of interpreting what it means
- A simple pair of a key and an associated collection of values. Key is usually a string. The database has no knowledge of the structure or meaning of the values.

Types of NoSQL: document databases

- Similar to a key-value store except that the document is "examinable" by the databases, so its content can be queried, and parts of it updated.
- Like a key-value store, but "document" goes further than "value". The document is structured, so specific elements can be manipulated separately

Types of NoSQL: column families

- Columns rather than rows are stored together on disk. Makes analysis faster, as less data is fetched. This is like automatic vertical partitioning.
- Data is grouped in "column groups/families" for efficiency reasons
- Related columns grouped together into 'families'.

Aggregate-oriented databases

- **Key-value, document store** and **column-family** are "aggregate-oriented- store business object in its entirety" databases (in Fowler's terminology)
- Pros:
  - entire aggregate of data is stored together (no need for transactions)
  - efficient storage on clusters / distributed databases
- Cons:
  - hard to analyse across subfields of aggregates
  - e.g. sum over products instead of orders

ACID vs BASE

ACID (Atomic, Consistent, Isolated, Durable)

vs

Base (Basically Available, Soft State, Eventual Consistency)

- Basically Available: This constraint states that the system does guarantee the availability of the data; there will be a response to any request. But data may be in an inconsistent or changing state.
- Soft state: The state of the system could change over time -even during times without input there may be changes going on due to 'eventual consistency'.
- Eventual consistency: The system will eventually become consistent once it stops receiving input. The data will propagate to everywhere it needs to, sooner or later, but the system will continue to receive input and is not checking the consistency of every transaction before it moves onto the next one.

Star schema